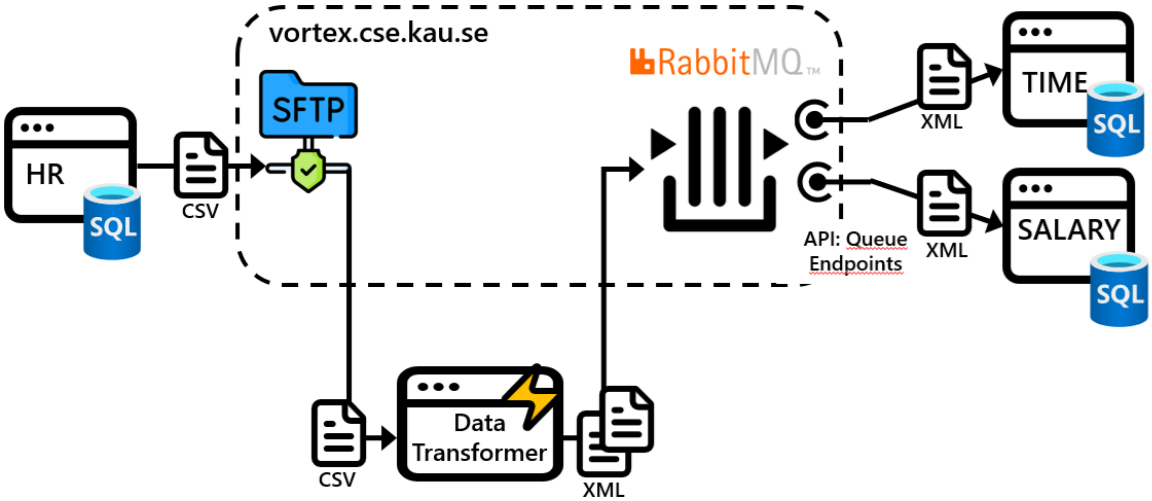
Labb 3 – Köhantering



I denna labb ska vi skicka våra transformerade format till en köhanterare (RabbitMQ) och skicka de två olika formaten till olika köer istället för till SFTP. Två applikationer (”TIME” och ”Salary”) ska konsumera meddelanden från sina respektive köer via RabbitMQ:s API. Projekten finns att ladda ner här:

- TIME: https://git.cs.kau.se/cajrolln/dvga25/-/blob/main/DVGA25_TIME.zip
- Salary: https://git.cs.kau.se/cajrolln/dvga25/-/blob/main/DVGA25_Salary.zip

Er uppgift är att implementera stöd för att skicka transformerade meddelanden till köer i RabbitMQ istället för till SFTP i Data Transformern. Ni skall dessutom komplettera funktionerna som läser meddelanden från respektive kö i applikationerna ”TIME” och ”Salary”. En detaljerad beskrivning av denna labb följer nedan. Använd RabbitMQ-manager för att se status på era köer m.m.: RabbitMQ Management. **Inloggningen till RabbitMQ är**: user: KaU-ID och lösenordet är: <ditt kauid>_rmq_password (t.ex. ”janedoe100_rmq_password”)

1. Ladda ner och konfigurera applikationerna ”TIME” och ”Salary”

Ladda ner projekten via länkarna ovan och packa upp dem till en lämplig katalog på er dator. Öppna respektive projekt i Visual Studio genom att dubbelklicka på solution-filerna (**DVGA25_TIME.sln** resp. **DVGA25_Salary.sln**).

Båda applikationerna har en ”databas” i form av en varsin xml-fil (**employee_time.xml** resp. **employee_salary.xml**) och dessa finns i samma kataloger som solution-filerna. I

programkoden förväntas dessa filer ligga under ”C:\Temp\” på era datorer. Vill ni byta sökväg så måste ni ändra i koden på ett par ställen. Vill ni använda den sökväg som redan är implementerad i programmen så behöver ni bara kopiera xml-filerna till ”C:\Temp\”.

När xml-filerna ligger på de sökvägar som är angivna i programkoden så kommer applikationerna att kunna starta och ser ut ungefär som följer:

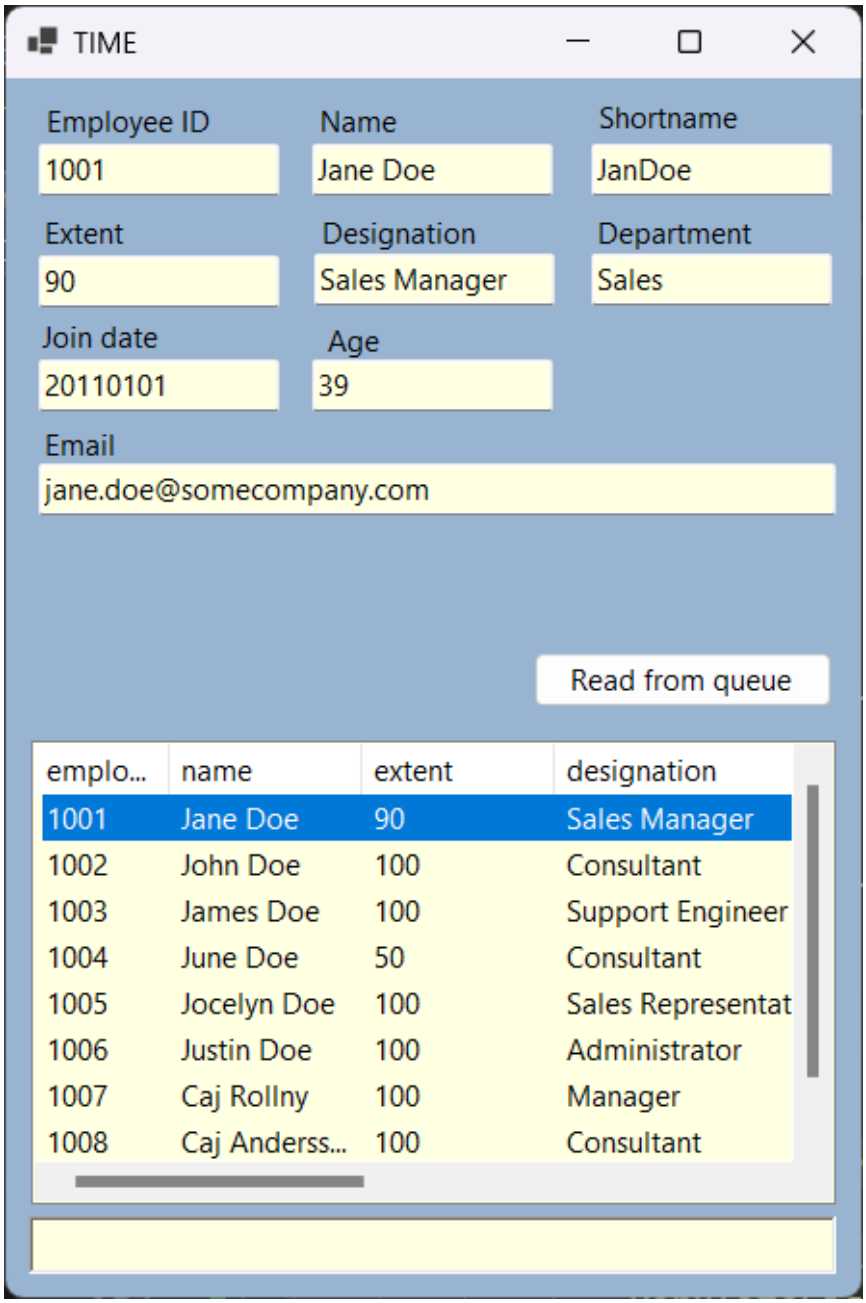


Fig. 1: Applikationen ”TIME”

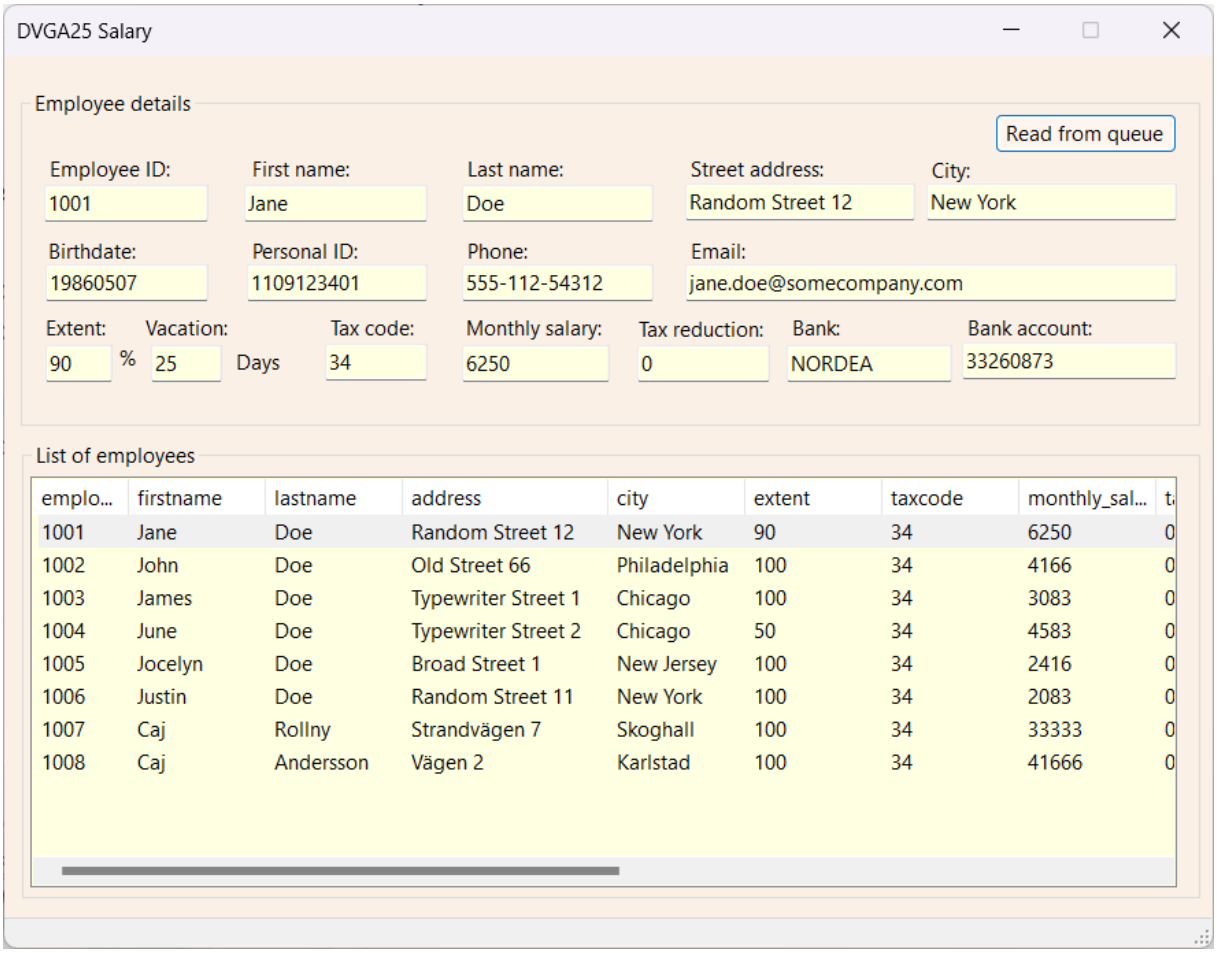
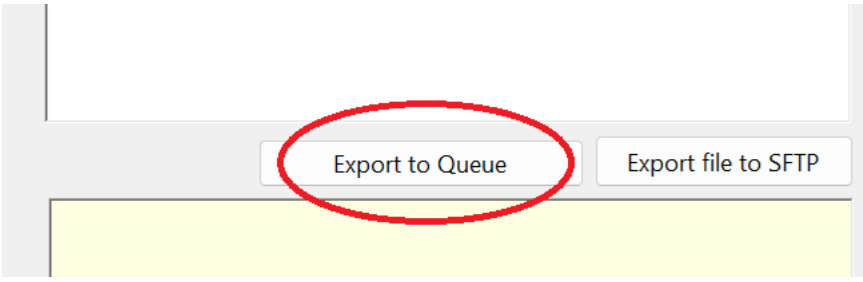


Fig. 2: Applikationen ”Salary”

2. Skicka (”publish”) respektive utfil till separata köer

Datatransformern skall nu skicka sina meddelanden till rätt köer via er personliga exchange på RabbitMQ-servern (vortex).

- Skapa en ny knapp och ge den ett lämpligt namn (t.ex. ”btnExportToQueue”) och lämplig text (t.ex. ”Export to Queue”). När du sedan dubbelklickar på denna knapp i designerläget så kommer en funktion att skapas som fångar klick-eventet för denna knapp.



- En funktion för att skicka meddelanden till kö tillhandahålls i klassfilen Producer.cs som finns här: https://git.cs.kau.se/cajrolln/dvga25/-/blob/main/Producer.cs
- Denna fil ska läggas till i Data-transformer-projektet. Du skall sedan anpassa funktionen ”PublishToQueue” så att den skickar till just din exchange och kö.
- Ett objekt skall skapas utifrån Producer-klassen: deklarera först en ”global” klassvariabel i Form1.cs. lägg den längs upp i klassen:

```
public partial class Form1 : Form
{
      private Producer producer;
```

- Instansiera sedan Producer-objektet i konstruktorn för Form1:

```
public Form1()
{
      producer = new Producer();
```

- Funktionen ”PublishToQueue” som finns i Producer-klassen skall sedan anropas när knappen ”Export to Queue” klickas. Kod för detta får ni nedan. Observera att en del av funktionens signatur måste ändras (”private void” skall ändras till ”private **async** void”).

```
private async void btnExportToQueue_Click(object sender, EventArgs e)
{
    try
    {
        //LAB3: ändra filsökväg vid behov:
        string fileName = @"C:\Temp\employee_salary.xml";

        if (File.Exists(fileName))
        {
            //LAB3: skicka meddelande till din kö för Salary,
            //ändra till dina uppgifter:
            await producer.PublishToQueue(fileName, "cajroll100_SALARY", "SALARY");
        }
        else
        {
            MessageBox.Show("Export error. File: " + fileName + " is not found.");
        }
        //LAB3: ändra filsökväg vid behov:
        fileName = @"C:\Temp\employee_time.xml";

        if (File.Exists(fileName))
        {
            //LAB3: skicka meddelande till din kö för Salary,
            //ändra till dina uppgifter:
            await producer.PublishToQueue(fileName, "cajroll100_TIME", "TIME");
        }
        else
        {
            MessageBox.Show("Export error. File: " + fileName + " is not found.");
        }

    }
    catch (Exception ex)
    {
        MessageBox.Show("Error: " + ex.Message);

    }

}
```

3. Ta emot (”subcribe”) meddelanden från respektive kö

Båda applikationerna (”TIME” och ”Salary”) innehåller en klass som heter ”Consumer” (se kod i ”Consumer.cs”). Denna klass innehåller en funktion ”Receive” som anropas när man klickar på knappen ”Receive from queue”. Det ni skall göra är att **komplettera koden i funktionen ”Receive” för båda applikationerna så att anropen går till rätt köer**. Leta efter kommentarer med texten ”LAB3” (se exempel i Fig. 3 ovan) om det är svårt att hitta rätt.

Redovisning

Denna skall redovisas för labbhandledare på följande sätt:

- Ändra på data för en person (alt. lägg till en ny person) i HR-applikationen och spara.
- Läs in den uppdaterade (employee_master.csv) filen till Data Transformern via SFTP
- Transformera till intermediate-format och de slutliga formaten för ”TIME” och ”Salary”.
- Exportera meddelandena till köerna för ”TIME” och ”Salary” och verifiera att meddelanden har kommit in till köerna (använd RabbitMQ-managern).
- Starta sedan TIME-applikationen och Salary-applikationen och läs in respektive meddelanden från kön.
- Verifiera att data för den nya personen nu finns i båda applikationerna och med korrekt data.